

クウハクスコープ 技術仕様書

transit-accessibility / 公共交通オープンデータチャレンジ2026 応募作品

2026-09-22 現在 / GitHub: K-hskw/transit-accessibility ブランチ `fix/network-and-isochrone` / 対象: 北海道室蘭市・道南バス GTFS-JP

この文書の目的：本プロジェクトの文脈を持たない読み手（生成AIを含む）が、コードを見ずに設計判断の可否を議論し、変更提案ができる状態になること。数値はすべて実測値。**2026年5月作成の引き継ぎ文書には、後述する誤りが含まれるため、矛盾する場合は本書を優先すること。**

1. 何をするアプリか

GTFSの時刻表から「いつ・どこから・どこへ行けるか」を計算し、**生活拠点に到達できない人口**を地図に可視化する。さらに増便・ダイヤ変更がその人口を何人減らすかを比較する。想定利用者は自治体の交通政策担当者。

1.1 中心概念：時間空白

既存の交通空白の把握は「バス停から半径1km圏外」のような**距離基準**が主流。本作は**時刻表由来の到達可能性**で定義する。同じ地点でも時間帯・曜日によって到達可能性が変わり、距離基準ではこれが見えない。

区分	定義	施策で変わるか
静的空白	徒歩圏（既定500m）にバス停が1つも無いメッシュ	変わらない（停留所新設が必要）
時間空白	バス停はあるが、制限時間内に拠点へ着けないメッシュ	変わる。本作の測定対象

2. データ

データ	内容	規模
GTFS-JP（道南バス）	stops / stop_times / trips / calendar / routes / shapes	停留所 2,329
bus_edges.csv（生成物）	便ごとの停留所間区間。全ダイヤ分を保持	115,406 行
walk_edges.csv（生成物）	停留所間300m以内の徒歩エッジ（双方向）	5,676 行
100mメッシュ人口	令和2年国勢調査・室蘭市（Meshcode, PopT, Pop65over）	3,079 メッシュ
facilities.csv	病院・スーパー等6種の施設POI（手動収集）	60 件

室蘭市 総人口 83,050人 / 65歳以上 30,617人（高齢化率 36.9%）。route_id は 108（うち高速バス等6を除いた102が市内路線）。

2.1 ダイヤ種別

`bus_edges.csv` は**全ダイヤ分**を持ち、平日／土曜／日祝の切替は実行時に `trip_id` で絞る。種別ごとにネットワークを作り直さない。

種別	便数	エッジ数	判定
平日	806	33,005	calendar.txt の monday～friday のいずれかが1
土曜	674	27,310	saturday が1
日祝	670	27,200	sunday が1

土曜と日祝は `trip_id` が異なるが時刻表の内容は同一（到着便数・時間帯分布が完全一致）。実データの性質であり不具合ではない。

`calendar_dates.txt`（特定日の運休・追加）は未対応で、祝日ダイヤは「日祝」のservice_idで近似している。

3. モジュール構成

```

service_calendar.py  (53行)  ダイヤ種別の判定。他モジュールに依存しない
    ↑
build_network.py     (132行)  GTFS → bus_edges.csv / walk_edges.csv (1回だけ実行)
transit_engine.py    (763行)  ★探索エンジン。アルゴリズム本体は47行
population.py        (196行)  メッシュ人口・施設。距離計算はnumpyベクトル化
    ↑
blank_area.py        (203行)  空白人口の算出 (population に依存)
prescription.py      (158行)  処方の評価 (engine/analyzer を引数で受け取る)
    ↑
app.py               (1327行)  Streamlit UI。上記すべてを使う

```

依存は一方向。`prescription.py` は engine を import せず引数で受け取るため単体で差し替え可能。

4. アルゴリズム

4.1 時刻依存ダイクストラ法 (`_dijkstra`)

通常の最短経路は辺の重みが固定だが、バスは**何時にその停留所へ着いたかで次の一手のコストが変わる**。8:15着なら8:20発に乗れる (待ち5分) が、8:25着なら9:00発まで待つ (待ち35分)。

実装の要点

- グラフの辺は抽象的な「区間」ではなく**便ごとの具体的な発車**。`bus_graph[stop] = [(dep_sec, arr_sec, to_stop, trip_id), ...]` を発車時刻順にソートして保持
- 緩和条件は `dep_sec >= current_time` のみ。**待ち時間は式に現れず、この制約から自動的に生じる**
- 最小化する量は距離ではなく**到着時刻**。優先度付きキューも時刻順
- 状態は停留所ではなく**(停留所, 直前の移動手段)**の組。`last_mode != "walk"` により**徒歩の連続を禁止**する。これが無いと300m徒歩エッジを連鎖して徒歩だけで市内を横断できてしまう
- 深夜便対応として時刻は24時超も秒で表現 (例 25:30 = 91800秒)

追い越しに強い：便を1本ずつ列挙しているため、急行が各停を追い越す場合も両方が候補に入り早く着く方が選ばれる。**FIFO性 (追い越しなし) の仮定を必要としない**。

注意：引き継ぎ文書の「急行便は自動除外しFIFO性を保持」は誤り。高速バスは経路探索に**使われている**。

`get_muroran_routes(exclude_highway=True)` が除外するのはUIの路線選択リストのみで、除外理由は「市内交通と性質が異なり廃止シミュレーションの対象として不適切」というモデル上の判断。アルゴリズムの正当性のためではない。

4.2 逆方向到達圏 (`calc_reverse_isochrone`)

向きを逆にし、「**指定時刻までに拠点へ着くための最も遅い出発時刻**」を最大化する。バスエッジを `to_stop → (arr_sec, dep_sec, from_stop)` で辿り、`arr_sec <= current_time` かつ `dep_sec >= earliest_time` を満たす便を緩和する。順方向と同じく徒歩の連続を禁止する。

なぜ重要か：空白人口は「全メッシュから拠点へ行けるか」なので、順方向だと3,079メッシュ分の探索が要る。逆方向なら**拠点から1回**で全メッシュ分が出る。3,079メッシュ×17時間帯の集計が0.10秒で終わるのはこのため。計算量だけでなく、問いの立て方の変更でもある。

4.3 空白人口の算出 (`BlankAreaAnalyzer`)

1. 事前計算：メッシュ重心×停留所のうち徒歩圏(500m)に入る組を保持 (19,408組)
2. 拠点から逆方向到達圏 → 各停留所の「最も遅い出発時刻」
3. メッシュごとに `max(停留所の最遅出発時刻 - そこまでの徒歩時間)`
4. それが (到着時刻 - 制限時間) 以上なら到達可能、未満なら空白

- 徒歩圏に停留所が1つも無いメッシュ = **静的空白** (対応表に現れないので自然に区別される)
- **メッシュ重心→最寄り停留所の徒歩区間はここで初めて入る**。`bus_edges/walk_edges` には停留所間の徒歩しか無く、「そもそもバス停まで歩けるか」は評価されていなかった

- 拠点IDが空・未知なら `ValueError`。存在しない停留所名を渡した事故が「空白人口＝総人口」として黙って通るのを防ぐため

5. シナリオ機構（処方の土台）

当初 `simulate_*` は改変エッジを内部で作った直後に前方ダイクストラを回し、**起点1停留所からの到達圏しか返さなかった**。一方、空白人口は逆方向で計算する。そのため「代替路線を引いたら空白人口が何人減るか」が計算できなかった。各シミュレーションを「改変後エッジを返す関数」と「探索する関数」に分離し、`scenario()` で一時適用できるようにした。

```
before = analyzer.diagnose(dest, t, limit)
with engine.scenario(*engine.edges_after_route_removal(rid)):
    after = analyzer.diagnose(dest, t, limit)  # 改変後のネットワークで診断
# 抜けるとネットワークとグラフキャッシュが復帰（例外時も）
```

改変関数	内容	徒歩エッジ
<code>edges_after_route_removal</code>	路線の廃止（単体/複数）	不変
<code>edges_after_stop_removal</code>	停留所の削除。路線は通過扱いで維持	変わる
<code>edges_after_frequency_reduction</code>	減便（half / interval / all）	不変
<code>edges_after_route_replacement</code>	路線廃止 + 新規路線の追加	不変
<code>edges_after_frequency_increase</code>	増便	不変
<code>edges_after_timetable_shift</code>	ダイヤシフト（便数不変・費用中立）	不変

`simulate_*` はこれらと呼ぶ薄いラップとして残っている（既存UIとの互換のため）。テストで両者の結果一致を検証しており、片方だけ直してずれる事故を検出できる。

5.1 増便の設計判断

停留所パターン（=経路と方向の組）ごとに、指定時間帯へ等間隔に便を挿入する。1路線に最大4パターンあるため、方向を無視すると「行きの便を複製して帰りに足す」不整合が起きる。

既存便の隙間を埋める方式は採らない。空白が深刻な時間帯ほど既存便が1本以下で隙間が定義できず、**最も増便したい路線に限って増便できなくなる**ため。運行が皆無の時間帯に新設する場合は `edges_after_route_replacement` を使う。

6. 発見・修正した重大な不具合

いずれも**分析結果を誤らせる種類のもの**。再発させないこと。

6.1 所要0分区間の欠落（最重要）

`build_network.py` が `travel_time > 0` の区間しか採用していなかった。GTFS-JPの時刻は分単位のため、近接停留所間は所要0分になる。

影響	値
除外されていた区間（平日）	9,760 / 115,415 (8.46%)
影響を受けた便	1,833 / 2,837
モデル上バスで到達不能だった停留所	148 → 0

消えていたのは すずらん団地・みどり団地・ちのみ団地 など、まさに空白人口を測りたい住宅地。**修正は `>= 0`**。時刻表の時刻は絶対値なので0秒エッジを残しても誤差は停留所あたり1分未満で、便の総所要時間には累積しない。時刻逆転（負値）は0件。

6.2 同じ不具合が削除シミュレーション内部にも

`simulate_stop_removal` の通過再接続が `arr_sec > dep_sec`（厳密）で区間を作っていた。**何も削除しなくても到達圏が 212 → 91 に壊れる**状態で、削除の影響を大きく過大評価していた。修正後、東室蘭駅東口の削除による到達不能は 132 → 2 箇所（当該停留所のみ）。

6.3 逆方向到達圏に徒歩の連続禁止が無かった

順方向にある制限が逆方向に無く、300m徒歩エッジを無制限に連鎖できていた。集客圏が18.2%過大（280 → 229停留所）。両方で移動ルールが異なると前後比較が成立しない。

6.4 対応しないモードで共通ボタンを押すとクラッシュ

サイドバーの「シミュレーション実行」を全モードで表示していたが、実行処理は5モードしか扱わず、他のモードで押すと `NameError: result_after` で落ちた。**ブランチ分岐前から存在**していた不具合。ボタンを対応モードのみに限定して修正。

7. 引き継ぎ文書（2026年5月）の誤り

古い資料を参照する場合、以下は**使用しないこと**。いずれも 6.1 の欠落ネットワーク上の数値。

項目	旧（誤）	現在（実測）
時間帯格差（室蘭駅前・60分）	13.2倍	27.6倍（5時台11 / 10時台304）
路線廃止の影響	42路線中5	102路線中5（最大72箇所）
減便 30%→40%	14 → 60箇所（急増）	76 → 109箇所（段階的）
主要ハブ削除	24箇所が孤立	2箇所のみ
デマンド交通の効果	人口 +81%	機能を削除（後述）
バス停通過の時間短縮	停車時間30秒の削減	誤り。停車時間は全118,252レコードで0秒

8. 現在の主要な計測結果

拠点 = 東室蘭駅東口、制限60分、徒歩500m・分速67m。

到着時刻	平日	土日祝
7時着	21,195（25.5%）	38,322（46.1%）
9時着	19,867（23.9%）	13,825
12時着	20,488	20,326
17時着	24,245	26,417

平日9時着の内訳は 静的空白 3,793人 + 時間空白 16,074人。**平日と土日は互いに素な時刻表なので、どちらの空白が大きいかは時間帯で入れ替わる**。大小関係を決め打ちしてはいけない（開発中に「休日は常に空白が多い」と仮定したテストが9時台で落ちた）。

8.1 処方と比較（休日7時着・基準38,322人）

施策	7時の削減	全日の削減	追加便	運行キロ増
増便 候補3路線すべて	0	10,595	12	247.6
増便 10絵鞆・工大線	0	8,847	1	20.3
シフト 八丁平すずかけ線	8,214	4,773	0	0.0
シフト 絵鞆循環線	0	-1,719	0	0.0

評価は必ず全時間帯で行うこと。ダイヤシフトは便を移すだけなので狙った時刻は必ず改善するが、便を抜いた時間帯は悪化する。単一時刻だけで測ると「費用ゼロで改善」に見え、**全日で純悪化する施策（絵鞆循環線の例）を無害と誤認する。**

対象路線を便数で選んではいけない。最多便の鉄北北口線は129停留所を通るが空白地域は0箇所で、増便しても1人も減らない。 `rank_routes_by_blank_coverage()` で空白地域の停留所数により順位づけする。

費用は**運行キロ（便数×路線長）を代理値**とする。実額は運転士人件費・委託契約で決まり手元に無いが、バス事業のコストは慣例的に円/実車走行キロで表され補助金算定も走行キロ基準のため、単価を掛ければ金額になる。路線長は停留所間の直線距離の和で、実道路距離より短く出る（`shapes.txt` は未使用）。

9. 性能

処理	修正前	現在	手段
時間帯別到達圏（18回）	約30秒	0.02秒	グラフのキャッシュ化
人口計算（60停留所）	5.24秒	0.011秒	numpyベクトル化（結果一致を確認）
施策比較（4施策）	6.43秒	1.50秒	路線長のキャッシュ化

探索本体は最初から**0.00秒**で、所要時間のほぼ全てはグラフ構築が占めていた。`iterrows` → `itertuples` の置換も併用。キャッシュは `bus_edges` / `walk_edges` 差し替え時に破棄する必要があり、`scenario()` と `set_day_type()` が面倒を見る。

10. テスト

合計975件。実行は各ファイルを直接 `python test_*.py`。

ファイル	件数	内容
test_all.py / _v2 / _v3	474 / 137 / 286	データ整合、到達圏、各シミュレーション、人口・施設連動
test_blank_area.py	24	空白判定の性質、500mメッシュ集約の保存則
test_day_type.py	13	ダイヤ切替、キャッシュ破棄、単調性
test_scenario.py	17	scenario の適用・復帰・例外安全、 <code>edges_after_*</code> と <code>simulate_*</code> の一致
test_prescription.py	25	候補順位づけ、増便・シフト、費用中立性、全日評価

重要な不変条件

- ・ **単調性**：便を増やせば到達範囲は広がる（土曜+平日 ≧ 各々）。減便率を上げれば空白は増える。時刻依存ダイクストラが壊れればここで検出できる
- ・ **一致性**：`edges_after_X` + 探索 の結果が `simulate_X` と一致する
- ・ **保存則**：500mメッシュ集約の前後で総人口・空白人口が保存される
- ・ **費用中立**：ダイヤシフトで総便数・総運行キロが変わらない
- ・ 静的空白は施策で変わらない（徒歩圏の問題なので）

11. 意図的に採らなかった選択

項目	判断と理由
デマンド交通	機能ごと削除 。機能過多を避けるため。実装上も、デマンドを徒歩エッジとして追加していたため「徒歩の連続禁止」が適用され、徒歩→デマンド／デマンド→徒歩ができない歪みがあった。復活は git 履歴から可能
勾配（Tobler式・DEM）	保留 。徒歩速度を一律半減（S=+0.20相当の最悪ケース）させても空白人口の増加は+9.1%にとどまる。全メッシュが急勾配という前提での上限値なので実際はより小さい。DEM取得の工数に見合うか未判断
250mメッシュ	500mを採用 。100mメッシュは1kmの10等分のため250m（4等分）には整数で割り切れない。標準メッシュ体系で100mから作れるのは500m（5×5セル）
空間軸の処方为主軸に	時間軸を主軸に 。「どこに路線を引くか」は2025年優秀賞「コミバスをつくろう！」と competing。診断で時間空白を示しながら処方が空間軸では主張の軸がずれる
3Dグラフィックの生成AI利用	不採用 。計算結果と一致しない図はEBPMツールの信頼性を損なう。柱の高さは計算値そのもの

12. 既知の限界

- `calendar_dates.txt` 未対応。祝日ダイヤは「日祝」のservice_idで近似
- 徒歩エッジ生成が停留所の総当たり ($2,329^2 = 542$ 万回のPythonループ)。大都市では実用にならない
- 費用代理の路線長が直線距離のため実道路距離より短い。 `shapes.txt` が未使用
- 施設データが手動収集の60件のみ。国土数値情報からの自動取得は未実装
- バス停削除時の徒歩再接続は、実測では到達数に影響しない（バス直通経路が支配的）。過度に重視しないこと
- アプリの3Dモードと処方モードが個別に診断を走らせるため計算が重複する（体感1秒程度）
- main へのマージは未実施。公開中のアプリには未反映

13. 操作リファレンス

```
# 起動
streamlit run app.py

# ネットワーク再生成（GTFSを差し替えたときのみ。全ダイヤ分を生成する）
python build_network.py

# 3Dマップ（Streamlit不要の単体HTML）を再生成 → docs/kuhaku-scope-muroran.html
python tools/build_map3d.py

# テスト
python test_all.py / test_all_v2.py / test_all_v3.py
python test_blank_area.py / test_day_type.py / test_scenario.py / test_prescription.py
```

アプリのモードは10種：到達圏のみ／路線廃止／バス停削除／減便／時間帯別到達圏／施設アクセス／代替路線追加／集客圏分析／時間空白診断（3D）／処方（改善施策）。全モードで平日・土曜・日祝を切り替えられる。

依存は pandas / numpy / streamlit / folium / streamlit-folium / pydeck の6つのみ（geopandas・shapely・networkx は未使用のため除去済み）。

transit-accessibility / K-hskw / 本書の数値は道南バス GTFS-JP（2026年4～9月ダイヤ）と令和2年国勢調査100mメッシュ人口による実測値。拠点・制限時間・徒歩条件を変えると値は変わる。